

DISEÑO E IMPLEMENTACIÓN DE ARQUITECTURA EMPRESARIAL

**Semana 01
Sesión 1**

Ing. Jair Darnley Vasquez Aguirre

- 1. Sílabo**
- 2. Otros Documentos**
 - Reglamento general
 - Políticas de diversidad, equidad e inclusión para el estudiante.
 - Reglamento de disciplina del estudiante
- 3. Grabación de la Semana 01 (Sesión 2)**



Recomendaciones

- 1. Llega temprano a todas las clases.**
- 2. Entrega todas tus actividades formativas.**
- 3. Participa constantemente en clase.**
- 4. Pregunta si tienes dudas.**
- 5. Revisa los anuncios.**
- 6. Comunícate con tu delegado(a)**
- 7. No te copies, hazlo tú mismo**



¿Qué crees que puede ocurrir si un equipo de programadores empieza a desarrollar un sistema sin ningún tipo de planificación?



¿Qué prefieres: terminar todo un proyecto y recién mostrarlo al cliente, o mostrar avances periódicamente para recibir opiniones? ¿Por qué?



¿Crees que es posible desarrollar un software de calidad sin seguir una metodología? ¿Por qué?



PROYECTOS DE DESARROLLO DE SOFTWARE

Semana 01
Sesión 2

Ing. Jair Darnley Vasquez Aguirre

Logro de Aprendizaje

Al término de la sesión, el estudiante conoce la estructuración de fases bajo los procesos RUP para elaborar el plan de un proyecto de software, utilizando herramientas colaborativas.



¿Conoces alguna metodología o marco de trabajo utilizado para desarrollar software? ¿Cuál?



¿Qué papel crees que tiene el cliente durante el desarrollo de un sistema?

¿Solo participa al inicio y al final o debería participar durante todo el proyecto? ¿Por qué?



PROFESIONAL



Para un profesional de Ingeniería de Sistemas, conocer estos principios facilita participar y liderar proyectos de software con una visión más estratégica, evitando depender únicamente de las habilidades técnicas de programación.

PERSONAL



Los principios de la gestión de proyectos de software también pueden aplicarse a la organización de actividades académicas, laborales y personales.

CONTENIDO

01

**Proyecto de Desarrollo
de Software**

02

RUP

03

Las 6 Mejores Prácticas de RUP

04

Actividad

1. Proyecto de Desarrollo de Software

Proyecto de Desarrollo de Software

Un proyecto de desarrollo de software es un esfuerzo temporal orientado a crear, modificar o mejorar un producto o sistema de software que satisfaga determinadas necesidades de usuarios u organizaciones.

- ✓ Los requisitos pueden cambiar durante el proyecto.
- ✓ La estimación de tiempo y costos puede ser compleja.
- ✓ La tecnología utilizada puede evolucionar durante el proyecto.
- ✓ El producto es principalmente intangible.

Proyecto de Desarrollo de Software

Características de un Proyecto de Software

- **Temporalidad:** Tiene inicio y finalización.
- **Complejidad:** Puede involucrar múltiples componentes técnicos.
- **Cambio:** Los requisitos pueden evolucionar.
- **Dependencia Humana:** El conocimiento del equipo es fundamental.
- **Recursos Limitados:** Cuenta con presupuesto, personas y tecnología limitada.

Restricciones de un Proyecto de Software

ALCANCE

Define qué se va a desarrollar y qué funcionalidades estarán incluidas.

Ejemplo:

El sistema permitirá registrar estudiantes y realizar matrículas.

Restricciones de un Proyecto de Software

TIEMPO

Determina cuándo debe completarse el proyecto.

Ejemplo:

El sistema debe estar disponible antes del inicio del semestre académico.

Restricciones de un Proyecto de Software

COSTO

Representa los recursos económicos necesarios para desarrollar el proyecto.

Ejemplo:

El presupuesto máximo disponible es de S/ 80 000.

2. RUP

RUP (Rational Unified Process) es un proceso de desarrollo de software creado originalmente por Rational Software y posteriormente adquirido por IBM. Su propósito es proporcionar un marco estructurado para desarrollar software de manera iterativa, incremental y orientada a casos de uso.

Características

- ✓ Desarrollo iterativo e incremental.
- ✓ Gestión de requisitos.
- ✓ Uso intensivo de UML para modelar el sistema.
- ✓ Arquitectura como elemento central.
- ✓ Gestión continua de riesgos.
- ✓ Verificación de la calidad durante todo el proyecto.
- ✓ Control de cambios y configuración.
- ✓ Definición clara de roles y responsabilidades.

Fases de RUP

Inicio



Determinar la visión, alcance y viabilidad.

Elaboración



Analizar requisitos, riesgos y arquitectura.

Fases de RUP

Construcción



Desarrollar y probar el sistema.

Transición



Entregar el sistema a los usuarios.

Averiguar sobre las actividades realizadas en cada fase RUP.
Plantear un ejemplo práctico.



3. Las 6 Mejores Prácticas de RUP

Las 6 Mejores Prácticas de RUP

Uno de los aportes más importantes de RUP son sus seis mejores prácticas de ingeniería de software.

Estás prácticas buscan reducir los problemas habituales de los proyectos de software.

Las 6 Mejores Prácticas de RUP

Desarrollo Iterativo

En lugar de intentar construir todo el sistema de una sola vez, RUP propone desarrollar el software mediante iteraciones.

Cada iteración produce un incremento del sistema que puede ser evaluado.

Las 6 Mejores Prácticas de RUP

Desarrollo Iterativo

Ejemplo:

Se está desarrollando un sistema de biblioteca. En lugar de implementar todo a la vez (usuarios, libros, préstamos, multas, reportes, notificaciones) se puede trabajar progresivamente.

Permite obtener retroalimentación temprana y detectar problemas antes de que el proyecto esté completamente terminado.

Las 6 Mejores Prácticas de RUP

Gestionar los Requisitos

Consiste en identificar, documentar, organizar, priorizar y controlar los requisitos del sistema. Los requisitos representan lo que el sistema debe hacer y las condiciones que debe cumplir.

Las 6 Mejores Prácticas de RUP

Gestionar los Requisitos

Ejemplo:

El sistema deberá permitir que el estudiante registre cursos disponibles.

El sistema deberá responder a una consulta en menos de tres segundos bajo condiciones normales de operación.

Las 6 Mejores Prácticas de RUP

Utilizar arquitectura basadas en componentes

Propone construir el software utilizando componentes reutilizables y una arquitectura bien definida.

Un componente representa una parte del sistema que proporciona determinadas funcionalidades.

Las 6 Mejores Prácticas de RUP

Modelar visualmente el software

RUP promueve el uso de modelos visuales, especialmente mediante UML (Unified Modeling Language).

Los modelos permiten representar diferentes perspectivas del sistema antes y durante la implementación.

Las 6 Mejores Prácticas de RUP

Modelar visualmente el software

- Diagrama de casos de uso.
- Diagrama de clases.
- Diagrama de secuencia.
- Diagrama de actividades.
- Diagrama de estados.
- Diagrama de componentes.
- Diagrama de despliegue.

Las 6 Mejores Prácticas de RUP

Verificar continuamente la calidad

RUP considera que la calidad no debe comprobarse solamente al final del proyecto.

Debe evaluarse durante todo el proceso.

Las 6 Mejores Prácticas de RUP

Verificar continuamente la calidad

- Pruebas unitarias.
- Pruebas de integración.
- Pruebas funcionales.
- Pruebas de sistema.
- Pruebas de aceptación.

Las 6 Mejores Prácticas de RUP

Gestionar los cambios

Consiste en controlar los cambios realizados durante el desarrollo.

- ✓ El cliente solicita nuevas funcionalidades.
- ✓ Se modifican requisitos.
- ✓ Se detectan errores.
- ✓ Cambian las tecnologías.
- ✓ Cambian las necesidades del negocio.

4. Actividad

Modelos del Proceso de Software

Averiguar 3 metodologías de desarrollo de software y sustentar. Considerar trabajar con ejemplos.



**¿TIENE
ALGUNA
PREGUNTA?**





¿Qué hemos
aprendido hoy?

¿Para qué servirá
lo aprendido?

Actividad Formativa S01

**Resolver la actividad
planteada y cargar en la
plataforma UTP+ Class**

CIERRE

¡Muchas Gracias!



Ing. Jair Darnley Vasquez Aguirre